



PERTEMUAN 5

## Methods dalam Java

Memahami cara mendeklarasikan, menggunakan, dan melakukan overloading metode untuk membangun program Java yang terstruktur, efisien, dan mudah dipelihara.



# Pengenalan Methods & Overloading



Metode adalah fondasi utama dalam pengembangan program Java yang besar dan terstruktur. Dengan metode, kita dapat memecah program kompleks menjadi bagian-bagian kecil yang mudah dikelola.

## Fasilitasi Desain

Metode memungkinkan desain program yang modular, sehingga setiap bagian memiliki tanggung jawab yang jelas dan terdefinisi.

## Implementasi & Pemeliharaan

Program besar lebih mudah diimplementasikan dan dipelihara ketika logikanya dibagi ke dalam metode-metode yang berfokus pada satu tugas.

## Method Overloading

Teknik overloading memungkinkan beberapa metode berbagi nama yang sama namun bekerja dengan tipe atau jumlah argumen yang berbeda.

# Program Unit Java

## Java API & Class Library

Java menyediakan Java Application Programming Interface (Java API) — sebuah koleksi kelas dan metode yang telah ditentukan sebelumnya, siap digunakan untuk mempercepat pengembangan aplikasi.

## Menggabungkan Metode Baru & Existing

Dalam pengembangan aplikasi Java, programmer tidak perlu membangun segalanya dari nol. Anda dapat:

- Menulis metode dan kelas baru sesuai kebutuhan spesifik
- Menggabungkannya dengan kelas dari Java API
- Memanfaatkan berbagai perpustakaan kelas pihak ketiga
- Menciptakan program yang kuat dan efisien

① Kemampuan menggabungkan komponen yang sudah ada dengan kode baru adalah kunci produktivitas pengembang Java yang profesional.

# Static Method, Static Variable & Math Class

Sebagian besar metode dijalankan sebagai respons terhadap panggilan metode pada objek tertentu. Namun, **static method** dapat dipanggil langsung melalui nama kelasnya tanpa perlu membuat objek terlebih dahulu.

## Konsep Static Method

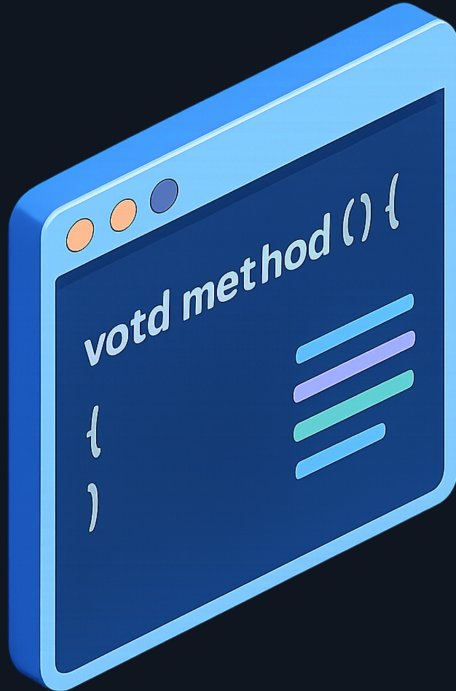
- Dipanggil menggunakan nama kelas, bukan nama objek
- Tidak memerlukan instansiasi objek
- Tersedia langsung dari kelas yang bersangkutan

## Contoh Math Class Methods

Kelas **Math** menyediakan berbagai metode matematis statis yang sangat berguna:

```
System.out.println(Math.sqrt(900.0));  
System.out.println(Math.sqrt(c + d * f));
```

Metode `Math.sqrt()` menerima nilai bertipe `double` dan mengembalikan akar kuadratnya.



# Declaring Method — Struktur Deklarasi Metode

Mendeklarasikan sebuah metode di Java mengikuti struktur yang baku dan konsisten. Pemahaman atas setiap bagian deklarasi sangat penting sebelum mulai menulis metode sendiri.

|   |                                                                                                                                                                  |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Return Type</b><br>Tipe data yang dikembalikan metode setelah eksekusi. Gunakan <code>void</code> jika metode tidak mengembalikan nilai apapun.               |
| 2 | <b>Method Name</b><br>Nama unik yang mengidentifikasi metode. Gunakan konvensi camelCase, misalnya <code>hitungLuas()</code> atau <code>tampilkanData()</code> . |
| 3 | <b>Parameter List</b><br>Daftar input yang diterima metode, terdiri dari tipe data dan nama variabel. Bisa kosong jika metode tidak memerlukan input.            |
| 4 | <b>Method Body</b><br>Blok kode di antara tanda kurung kurawal <code>{ }</code> yang berisi instruksi-instruksi yang akan dijalankan saat metode dipanggil.      |

# Deklarasi dan Penggunaan Method

Ada **tiga cara** utama untuk memanggil metode di Java, masing-masing digunakan dalam konteks yang berbeda:

## 1 Panggilan Langsung (Kelas yang Sama)

Menggunakan nama metode secara langsung untuk memanggil metode lain dalam kelas yang sama. Contoh: `maksimum(number1, number2, number3)`

## 2 Panggilan Melalui Objek (Non-Static)

Menggunakan nama variabel diikuti titik (.) dan nama metode. Metode non-statis disebut juga **instance method**. Contoh: `input.nextDouble()`

## 3 Panggilan Melalui Nama Kelas (Static)

Menggunakan nama kelas diikuti titik (.) untuk memanggil metode statis. Contoh: `Math.sqrt(900.0)`

# Pemanggilan Method & Aktivasi Stack

## Struktur Data Stack

Untuk memahami bagaimana Java mengelola pemanggilan metode, kita perlu memahami struktur data **stack (tumpukan)**. Stack bekerja dengan prinsip **Last-In, First-Out (LIFO)** — elemen terakhir yang dimasukkan adalah yang pertama dikeluarkan.

Bayangkan stack seperti tumpukan piring:

- **Push**: meletakkan piring baru di bagian atas tumpukan
- **Pop**: mengambil piring dari bagian atas tumpukan



## Prinsip LIFO

Item terakhir yang dimasukkan (pushed) ke dalam stack adalah item pertama yang dikeluarkan (popped). Prinsip inilah yang digunakan Java untuk mengelola urutan eksekusi metode secara teratur dan dapat diprediksi.

# Method Call Stack

Method Call Stack (tumpukan panggilan metode) adalah salah satu mekanisme paling penting yang harus dipahami oleh setiap mahasiswa ilmu komputer. Struktur data ini bekerja "**di belakang layar**" untuk mendukung seluruh mekanisme pemanggilan dan pengembalian metode dalam Java.



## Mendukung Panggilan & Pengembalian

Setiap kali sebuah metode dipanggil, informasi tentang panggilan tersebut ditumpuk pada call stack. Ketika metode selesai, informasi tersebut diambil kembali untuk melanjutkan eksekusi pemanggil.

## Mengelola Variabel Lokal

Call stack juga bertanggung jawab untuk pembuatan, pemeliharaan, dan penghancuran variabel lokal setiap metode yang dipanggil secara otomatis.

## Perilaku LIFO

Sifat Last-In, First-Out pada stack memastikan bahwa setiap metode selalu kembali ke pemanggil yang tepat, menjaga alur eksekusi program tetap benar dan teratur.

# Stack Frame (Bingkai Tumpukan)

Setiap kali sebuah metode memanggil metode lain, sebuah entri baru — yang disebut **stack frame** atau **activation record** — didorong ke atas call stack.

## Apa itu Stack Frame?

Stack frame adalah blok memori yang berisi semua informasi yang dibutuhkan metode untuk berjalan, termasuk:

- Alamat pengirim (return address)
- Parameter yang diterima metode
- Variabel lokal metode

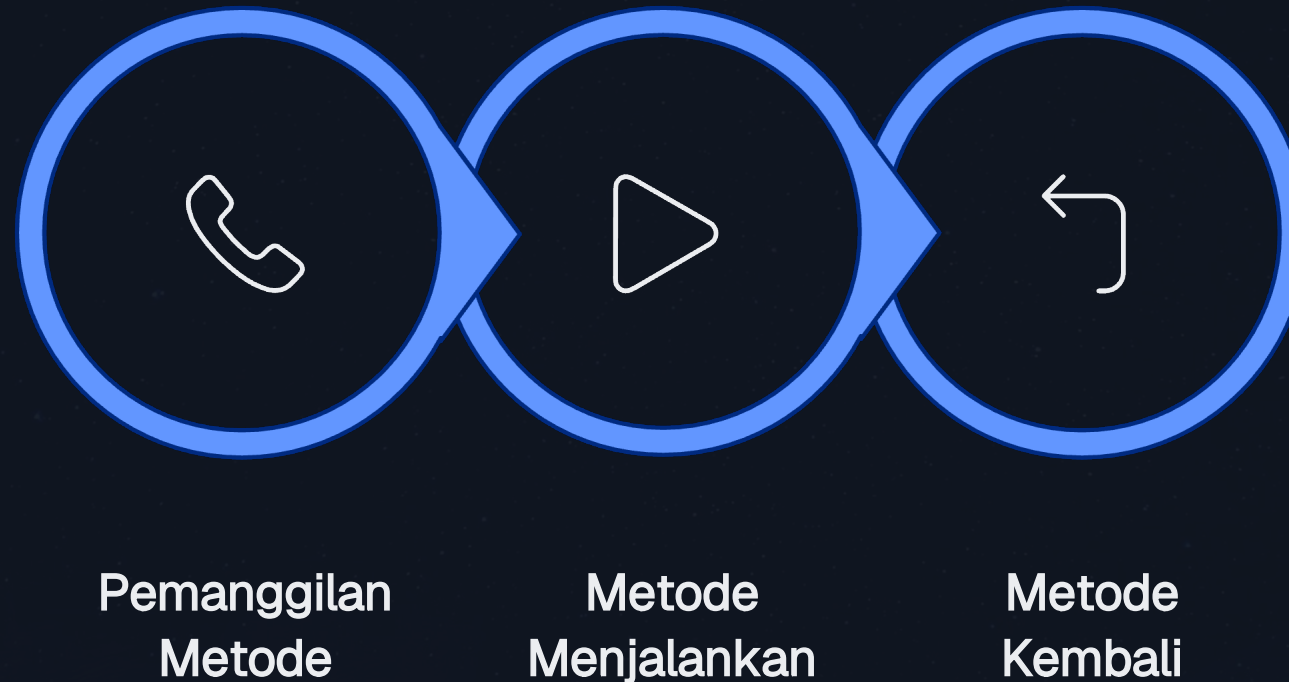
## Bagaimana Cara Kerjanya?

Ketika rantai pemanggilan metode terjadi (A memanggil B, B memanggil C), setiap metode mendorong frame-nya ke stack. Ketika C selesai, frame-nya diambil, dan kontrol kembali ke B dengan alamat yang tersimpan di stack frame B.

- Stack frame memungkinkan Java melacak rantai pemanggilan metode yang panjang sekalipun secara otomatis dan akurat.

# Local Variable dan Stack Frame

Stack frame memiliki peran krusial dalam pengelolaan **variabel lokal** setiap metode. Variabel lokal harus ada selama metode dieksekusi dan harus tetap aktif selama metode tersebut memanggil metode lain.



Bingkai tumpukan metode adalah tempat penyimpanan yang sempurna untuk variabel lokal karena siklus hidupnya selaras persis dengan siklus hidup metode itu sendiri — tercipta saat metode dipanggil dan lenyap begitu metode mengembalikan nilai.

# Stack Overflow

## Terlalu banyak panggilan = Stack Overflow!

Memori komputer bersifat terbatas. Ketika jumlah panggilan metode melebihi kapasitas yang tersedia untuk menyimpan activation record pada call stack, terjadilah error fatal yang dikenal sebagai **Stack Overflow**.

### Penyebab Utama

- **Rekursi tak terbatas** — metode memanggil dirinya sendiri tanpa kondisi berhenti yang valid
- Rantai pemanggilan metode yang terlalu dalam
- Alokasi variabel lokal berukuran sangat besar di dalam stack

### Dampak & Pencegahan

- Program akan berhenti secara mendadak dengan pesan error
- Java melemparkan `StackOverflowError`
- Selalu pastikan rekursi memiliki **base case** yang jelas
- Batasi kedalaman pemanggilan metode yang bersarang



# Argument Promotion dan Casting

**Argument Promotion** adalah fitur Java yang secara otomatis mengkonversi nilai argumen ke tipe yang lebih tinggi (wider type) jika tipe argumen yang diberikan berbeda dengan yang diharapkan metode — selama konversi tersebut aman dan tidak kehilangan data.

## Contoh Promotion Otomatis

```
System.out.println(Math.sqrt(4));
```

Meskipun `Math.sqrt()` mengharuskan argumen bertipe `double`, Java secara otomatis mempromosikan nilai `int 4` menjadi `double 4.0`.

## Hierarki Promotion

Java mengikuti hierarki tipe data dari yang terkecil ke terbesar:

`byte → short → int → long → float → double`

Konversi ke arah yang berlawanan (dari tipe lebih besar ke lebih kecil) memerlukan **explicit casting** dan berisiko kehilangan presisi data.

# Java API Packages

Java API diorganisir ke dalam **packages** — kelompok kelas yang saling berkaitan. Setiap package menyediakan fungsionalitas khusus yang dapat diimpor dan digunakan dalam program Anda.



## java.lang

Package inti Java yang diimpor secara otomatis. Berisi kelas fundamental seperti `String`, `Math`, `Object`, dan `System`.



## java.io

Menyediakan kelas untuk operasi input/output, termasuk membaca dan menulis file serta stream data.



## java.util

Berisi kelas utilitas seperti `Scanner`, `ArrayList`, `Random`, dan berbagai struktur data koleksi lainnya.



## java.net

Berisi kelas untuk pemrograman jaringan, termasuk koneksi internet, socket, dan URL handling.

# Case Study 1 — MaximumFinder

Studi kasus pertama mengimplementasikan metode untuk mencari nilai maksimum dari tiga bilangan desimal. Program ini mendemonstrasikan cara mendeklarasikan metode dengan parameter dan nilai kembalian.

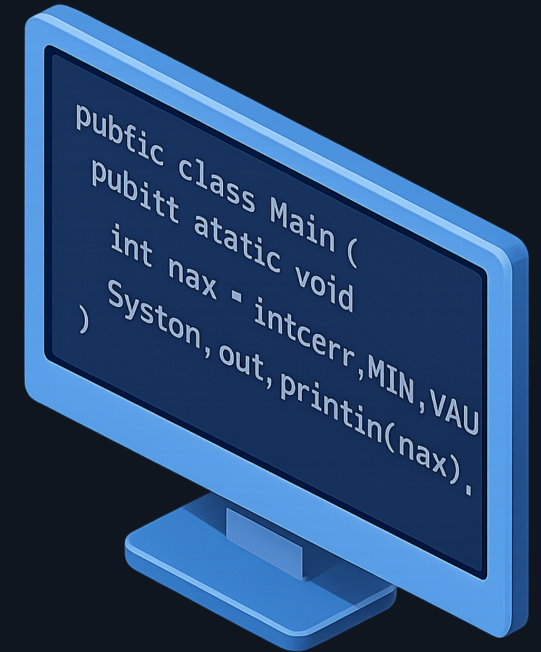
## Tujuan Program

Membuat metode `maximum()` yang menerima tiga nilai `double` sebagai argumen dan mengembalikan nilai terbesar di antara ketiganya menggunakan logika perbandingan.

## Konsep yang Diterapkan

- Deklarasi metode dengan tipe kembalian `double`
- Penggunaan tiga parameter bernama `x`, `y`, `z`
- Memanggil `Math.max()` dari Java API di dalam metode
- Memanggil metode dari `main()` menggunakan nama langsung

📄 Studi kasus ini memperlihatkan bagaimana metode membuat kode lebih bersih, dapat digunakan kembali, dan mudah diuji secara terpisah.



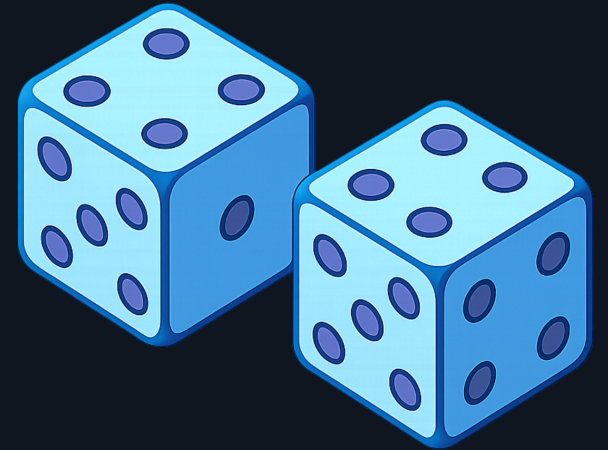
# Output & Alur Program MaximumFinder



Program MaximumFinder mengalir secara linear: pengguna memasukkan tiga angka, program memanggil metode `maximum()`, logika perbandingan berjalan di dalam metode, dan hasilnya ditampilkan kembali ke layar — memperlihatkan prinsip enkapsulasi logika dalam sebuah metode.

# Case Study 2 — Permainan Craps

Anda melempar dua dadu. Setiap dadu memiliki enam wajah berisi bintik satu hingga enam. Jika jumlah bintik pada lemparan pertama adalah 7 atau 11, Anda **menang**. Jika jumlahnya 2, 3, atau 12 (disebut "craps"), Anda **kalah**. Jika jumlahnya lain, angka itu menjadi "poin" Anda — terus lempar hingga mencetak poin yang sama untuk menang, atau lempar angka 7 untuk kalah.



## Menang Langsung

Lemparan pertama menghasilkan 7 atau 11

## Kalah Langsung

Lemparan pertama menghasilkan 2, 3, atau 12

## Tentukan Poin

Hasil lain (4,5,6,8,9,10) menjadi target poin

# Implementasi Program Craps

Program Craps menggunakan metode dan `enum` untuk mensimulasikan logika permainan dadu secara terstruktur. Konsep-konsep Java yang diterapkan meliputi:

1

## Random Number Generation

Menggunakan kelas `Random` dari `java.util` untuk men-generate angka acak antara 1–6 yang mensimulasikan lemparan dadu.

2

## Enum untuk Status Game

Menggunakan tipe `enum` dengan nilai `CONTINUE`, `WON`, dan `LOST` untuk melacak status permainan secara jelas dan type-safe.

3

## Loop dan Kondisi

Menggunakan perulangan `do-while` untuk terus melempar dadu hingga pemain menang atau kalah berdasarkan kondisi yang telah ditetapkan.

4

## Modular Method

Metode `rollDice()` dipisahkan untuk mengelola satu lemparan dadu, membuat kode utama lebih bersih dan logika mudah diuji.

# Skop dari Deklarasi (Scope)

**Scope** atau ruang lingkup deklarasi menentukan di bagian mana dari program sebuah variabel atau metode dapat diakses dan digunakan. Memahami scope adalah kunci untuk menghindari bug dan konflik nama dalam program Java.

## 1 Scope Parameter Metode

Ruang lingkup deklarasi parameter adalah **seluruh isi metode** di mana deklarasi parameter tersebut muncul — tidak dapat diakses dari luar metode.

## 2 Scope Variabel Lokal

Berlaku dari titik deklarasi hingga akhir blok `{ }` tempat variabel tersebut dideklarasikan. Di luar blok itu, variabel tidak dikenali.

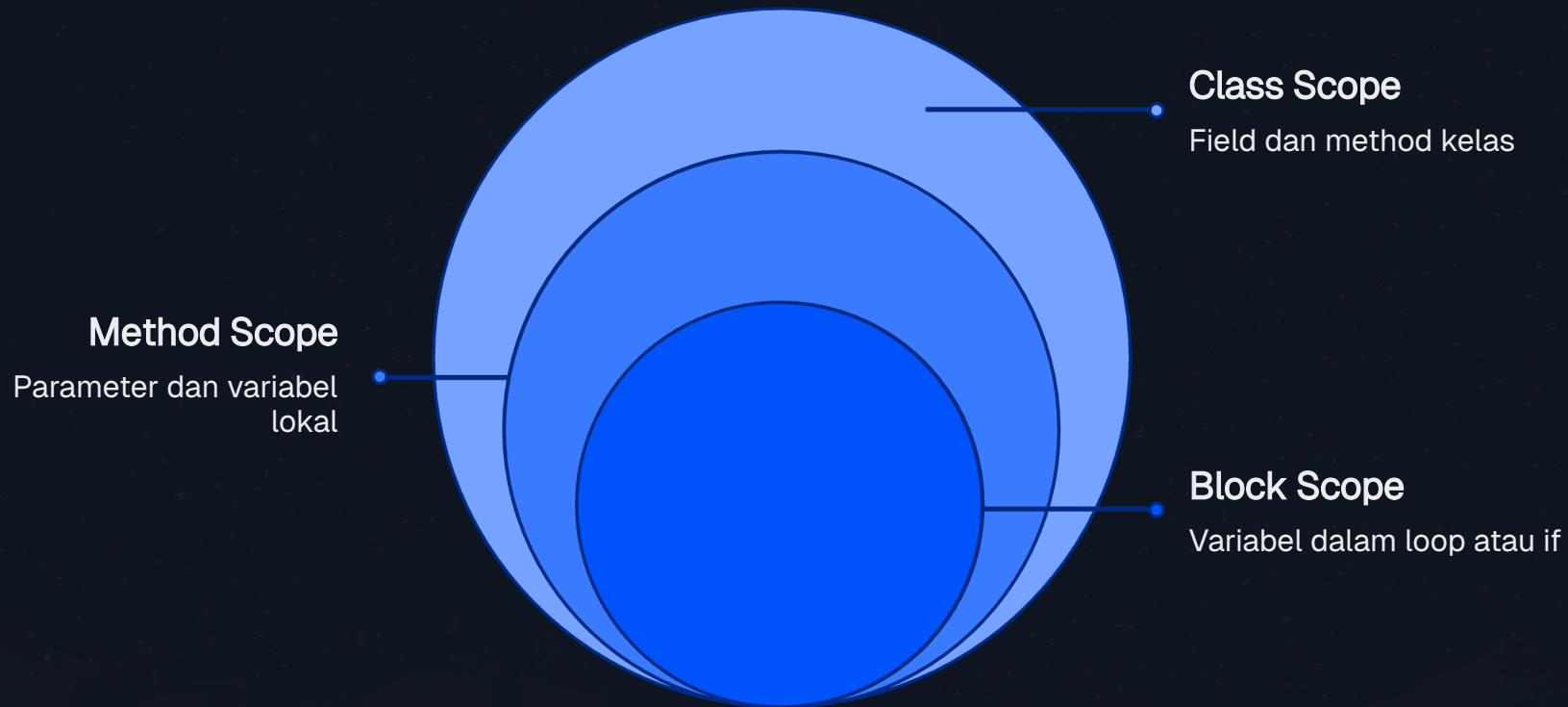
## 3 Scope Variabel dalam Loop For

Variabel yang dideklarasikan di header `for` hanya berlaku di dalam badan pernyataan `for` dan ekspresi header-nya saja.

## 4 Scope Field & Metode Kelas

Metode dan field (atribut) kelas memiliki scope yang mencakup **seluruh tubuh kelas** — dapat diakses oleh semua metode di dalam kelas yang sama.

# Visualisasi Scope dalam Program Java

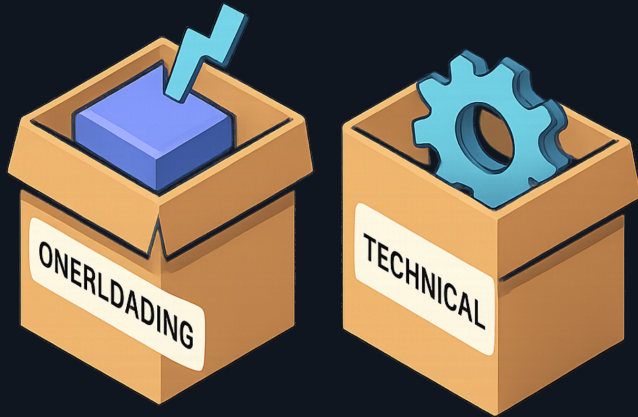


Scope bersifat bertingkat seperti lingkaran bersarang. Scope yang lebih dalam (inner scope) dapat mengakses variabel dari scope yang lebih luar, namun tidak sebaliknya. Memahami hierarki scope ini mencegah kesalahan seperti variabel tidak dikenali atau nilai yang tidak terduga.

 **Tips:** Selalu deklarasikan variabel di scope tersempit yang memungkinkan untuk menjaga kode tetap bersih dan menghindari efek samping yang tidak disengaja.

# Method Overloading

Java memungkinkan beberapa metode dalam satu kelas memiliki **nama yang sama**, asalkan daftar parameter mereka berbeda. Teknik ini disebut **method overloading** dan merupakan salah satu fitur terkuat dalam pemrograman berorientasi objek.



## Aturan Overloading

Metode yang di-overload harus berbeda dalam **jumlah**, **tipe**, atau **urutan** parameternya. Perbedaan hanya pada return type saja tidak cukup.

## Seleksi Kompiler

Saat metode overloaded dipanggil, kompiler secara otomatis memilih versi yang paling cocok berdasarkan argumen yang diberikan pada saat pemanggilan.

## Tujuan Penggunaan

Overloading membuat API lebih intuitif — programmer cukup menghafal satu nama metode untuk tugas serupa, meski digunakan dengan tipe data yang berbeda-beda.

# Mendeklarasikan Overloaded Methods

Berikut adalah contoh konkret bagaimana metode yang di-overload dideklarasikan. Ketiga metode di bawah memiliki nama `tambah()` namun menerima tipe argumen yang berbeda:

## Versi Integer

```
int tambah(int a, int b) {  
    return a + b;  
}
```

## Versi Double

```
double tambah(double a, double b) {  
    return a + b;  
}
```

## Versi Tiga Parameter

```
int tambah(int a, int b, int c) {  
    return a + b + c;  
}
```

## Bagaimana Kompiler Memilih?

Kompiler mencocokkan panggilan metode dengan versi yang tepat berdasarkan argumen:

- `tambah(3, 5)` → memanggil versi `int`
- `tambah(3.14, 2.71)` → memanggil versi `double`
- `tambah(1, 2, 3)` → memanggil versi tiga parameter

📌 Semua versi dapat hidup berdampingan dalam satu kelas tanpa konflik.

# Membedakan Overloaded Methods

Kompiler Java membedakan metode yang di-overload melalui sesuatu yang disebut **method signature** — kombinasi dari nama metode beserta jumlah, tipe, dan urutan parameternya. *Return type tidak termasuk dalam signature.*

## Contoh Signature Berbeda

```
void method1(int a, float b)
```

Kompiler membaca ini sebagai: "method1 dari int dan float"

```
void method1(float a, int b)
```

Kompiler membaca ini sebagai: "method1 dari float dan int"

 Urutan tipe parameter sangat penting! Kedua deklarasi di atas dianggap BERBEDA oleh kompiler.

## Komponen Method Signature

- ☒ Nama metode
- ☒ Jumlah parameter
- ☒ Tipe setiap parameter
- ☒ Urutan tipe parameter
- ☒ Return type (TIDAK termasuk)
- ☒ Nama variabel parameter (TIDAK termasuk)

# Return Types pada Overloaded Methods

Penting untuk dipahami: **pemanggilan metode tidak dapat dibedakan hanya berdasarkan return type-nya saja**. Jika dua metode memiliki signature yang identik namun return type berbeda, kompiler akan mengeluarkan *pesan error*.

## ✗ Tidak Valid — Hanya Beda Return Type

`int hitungLuas(int p, int l)` dan `double hitungLuas(int p, int l)` akan menyebabkan error karena signature-nya identik — kompiler tidak bisa membedakan keduanya.

## ✓ Valid — Beda Parameter + Beda Return Type

`int hitungLuas(int p, int l)` dan `double hitungLuas(double p, double l)` adalah valid karena tipe parameter berbeda, sekaligus return type-nya juga boleh berbeda.

## ✓ Valid — Jumlah Parameter Berbeda

Overloaded methods tidak harus memiliki jumlah parameter yang sama. `cetak(String s)` dan `cetak(String s, int kali)` keduanya valid dalam satu kelas.

📌 Kunci overloading: bedakan metode dari **daftar parameter**, bukan dari return type-nya.